Real-world (one reusable session) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

A `requests.Session()` object holds state (headers, cookies, and connection pooling) across multiple calls. Setting `session.headers.update({...})` merges your headers into every request made through `session.get(...)`, `session.post(...)`, and so on — you never repeat them per-call again.

There's a real efficiency reason too, not just cleanliness: a `Session` reuses the underlying TCP connection across calls to the same host, instead of opening a fresh one every time — genuinely faster for multiple calls to the same API.

The exact pieces:

- **`requests.Session()`** — construct once, reuse everywhere; don't create a new one per call, that defeats the whole point.
- **`session.headers.update(dict)`** — merges into the session's persistent headers dict; call it once, right after creating the session.
- **`session.get(url)`** / **`session.post(url, ...)`** — identical signatures to the module-level `requests.get` / `requests.post`, just bound to the session's state.
- **Confirming it actually worked** — inspect `session.headers["Authorization"]` after setting it, and/or check `response.request.headers` on one of the responses you get back, to see the header that was actually sent.

Sketch how you'd rewrite 3 separate `requests.get(url, headers=headers)` calls using one shared session, before checking Hint 2.

Think about who else might use this session besides the code you're writing right now. If a `Session` object gets shared across threads — say, a web server handling multiple requests at once, each one calling out to the same API — mutating `session.headers` from one thread while another thread is mid-request can cause subtle, hard-to-reproduce bugs, because `Session` was never designed to

have its state changed concurrently. The safe pattern is: configure a session's headers once, right after creating it, and treat it as read-only after that — never mutate a shared session's headers per-request from different call sites.

The extra piece:

- **Per-request headers, when you genuinely need one call to differ** — `session.get(url, headers={"X-Extra": "..."})` merges just for that one call, without touching the session's persistent headers at all; this is the safe way to vary one call without risking the mutate-a-shared-session problem above.

Hint 2 — The plan, and almost the whole thing

```
session = requests.Session()
session.headers.update({"Authorization": "Bearer fake-token"})

for url in [url1, url2, url3]:
    response = session.get(url)
    print(url, "->", response.status_code)

print(session.headers["Authorization"])
```

```
# session_reuse_practice.py
import requests

session = requests.Session()
session.headers.update({"Authorization": "Bearer fake-token"})

urls = ["https://api.github.com", "https://api.github.com/zen"]

for url in urls:
    response = session.get(url)
    print(f"{url} -> {response.status_code}")
```

What's missing: confirming the header was actually *sent*, not just set. Write that check yourself — look at `response.request.headers` on one of the responses — then compare against the Solution. Notice nothing about the loop needs to know or care about the header — that's the entire point of the session: the calling code gets simpler, not more complicated, as you add more calls. This factory-function shape is exactly what the Build Task's `http_client.py` uses — one module-level `Session`, configured once.

Full solution: Show me the solution